

Recall – תסריט מצגת • 9 דקות

21 שלבים • הדמו מסומן בכתום • הזמן בצד שמאל הוא היעד המצטבר

נקודות ביקורת: דפדפן ב-1:35 • חוזרת למצגת ב-5:10 • Architecture ב-5:35 מסלול שקפים: 1→2→3→4→5→6[דפדפן]→↓↓→8→9→10→↓↓→12→13→14→15→16 (מדלגים על 7 ו-11)			
0:20	0:20	1 About Me – שקף 1	1
0:40	0:20	2 Recall – שקף 2	2
1:05	0:25	3+4 Problem – שקפים 3 ו-4	3
1:25	0:20	5 Manual Way – שקף 5	4
1:35	0:10	6 What If? – שקף 6	5
1:55	0:20	6 DEMO 1 • הוק – נושא מהקהל	6
2:15	0:20	7 DEMO 2 • Add material + הכרות	7
2:45	0:30	8 DEMO 3 • generation	8
3:40	0:55	9 DEMO 4 • האדי עונה	9
4:10	0:30	10 DEMO 5 • feedback + התאריך	10
4:35	0:25	11 DEMO 6 • card שני – הניגוד	11
4:55	0:20	12 DEMO 7 • Progress	12
5:10	0:15	13 DEMO 8 • Grafana	13
5:20	0:10	14 Email – שקף 8	14
5:35	0:15	15 SM-2 – שקף 9	15
6:55	1:20	16 Architecture – שקף 10	16
7:35	0:40	17 Observability – שקף 12	17
7:50	0:15	18 Delivery – שקף 13	18
8:30	0:40	19 Testing – שקף 14	19
8:45	0:15	20 Working with AI – שקף 15	20
9:00	0:15	21 Close – שקף 16	21

1 • About Me – שקף 1 עד 0:20 • 0:20

מול הקהל, לא להסתכל על המסך.

שלום, אני שהד ריאן – בוגרת מדעי המחשב בחיפה, ובוגרת בוטקאמפ FURSA ב-AI וב-DevOps. היום אציג את פרויקט הגמר שלי, **Recall**. ולא חיפשתי נושא לפרויקט – **הייתה לי בעיה אמיתית בקורס**, ורציתי משהו שיעזור לי כבר עכשיו.

↓ מעבר

2 • Recall – שקף 2 עד 0:20 • 0:40

שקף Recall. לא להקריא את המספרים למטה (3 MCP tools, 8 Services...) – הם שם בשביל שיסתכלו, לא בשביליך.

Recall זה AI study tutor שעונה לי על שתי שאלות: **מה לחזור – ומתי**. נותנים לו חומר, הוא הופך אותו ל-**cards**, בוחן אותי – בטקסט או בקול – **מדרג את התשובה ומסביר מה היה חסר**, וקובע מתי אראה את ה-card הזה שוב.

↓ מעבר

3 • Problem • 3+4 – שקפים 3 ו-4 עד 0:25 • 1:05

שקף 3. במילה "עקומת השכחה" – ↓ לשקף 4, ולהצביע.

בבוטקאמפ למדנו נושא חדש כל שבוע – **Terraform, Kubernetes, Docker, AWS, Python**. בהתחלה זכרתי, ועם הזמן הידע התחיל להיעלם.

↓ זו **עקומת השכחה** – בלי חזרה, הזיכרון יורד משמעותית כבר אחרי יום.

ואז הבנתי: **הבעיה לא הייתה ללמוד – אלא לזכור**.

↓ מעבר

4 • Manual Way • 5 – שקף 5 עד 0:20 • 1:25

לעצור רגע על "cards 200".

ואם עושים את זה ידנית עם flashcards – צריך לא רק ללמוד, אלא **לנהל את כל התהליך**: לכתוב שאלות, לבדוק תשובות, לתת לעצמנו ציון, ולהחליט מתי לחזור.

עם **cards 200** זה כבר הופך לעבודה בפני עצמה – במקום ללמוד.

↓ מעבר

אז שאלתי: מה אם המערכת תעשה את כל זה בשבילי?

בואו נראה חי.

מעבר Alt+Tab לטאב 1. שנייה של שקט. △ המצגת נשארת על שקף 6. בסוף הדמו – ↓ פעמיים → שקף 8.

6 DEMO 1 • הוק – נושא מהקהל עד 1:55 • 0:20 DEMO

טאב 1 פתוח. להסתובב אל הקהל ולחכות. לא לדבר לתוך השקט.

זו המערכת – רצה עכשיו על ה-Kubernetes cluster שלי, לא local.

ולפני שאני מסבירה עליה משהו – תזרקו לי נושא שתמצאו ללמוד.

אם נתקע

אין תשובה תוך שתי שניות: *אז ניקח Kubernetes* – ולהמשיך מיד.

מעבר לחוץ Add material.

7 DEMO 2 • Add material + הכרות עד 2:15 • 0:20 DEMO

להקליד את הנושא, לחוץ Create. תוך כדי ההקלדה – להצביע על שלושת ה-tabs.

מעולה. אני נכנסת ל-Add material, כותבת את הנושא, ויוצרת.

להצביע והמערכת כולה בנויה משלושה חלקים: Add material – מביאים חומר. Study – לומדים.

Progress – רואים איפה אני עומדת.

מעבר ה-generation רץ.

8 DEMO 3 • generation עד 2:45 • 0:30 DEMO

לדבר לקהל, לא להסתכל על המסך. הטקסט נכתב בדיוק לאורך ההמתנה.

ובזמן שזה רץ – מה קורה מאחורה: ה-Agent קורא את החומר וכותב את השאלות בעצמו. הוא

מנסה לייצר שאלות שבודקות אם הבנתי, לא שינון.

כשמופיע וזהו – ה-deck מוכן. השאלות האלה לא היו קיימות לפני דקה.

אם נתקע

תקוע מעל 40 שניות: *בואו לא נחכה* → deck מוכן מראש.

מעבר Study → לפתוח את ה-deck החדש.

עד 0:55 • 3:40

DEMO

9 DEMO 4 • האדי עונה

9

לפתוח את ה-deck, להקריא את השאלה. לתת להאדי לענות – ולא למלא את השקט. לשלוח.

אני נכנסת ל-Study ופותחת אותו – כל ה-cards זמינים מיד, הם חדשים.

תראו את השאלה הזאת. **להקריא** האדי – תענה לנו.

זמן שהוא עונה ושימו לב שהוא צריך לשלוח מהראש – אין לו את התשובה מולו. זה **active recall**.

אם נתקע

נתקע: *תגיד מה שאתה זוכר, גם חלקי.*

מעבר לשלוח.

עד 0:30 • 4:10

DEMO

10 DEMO 5 • feedback + התאריך

10

להצביע לפי הסדר: score → התשובה המלאה → מה היה חסר → ואז למטה, next review.

והוא לא אומר "נכון" או "לא נכון": **להצביע** הוא נותן score, מראה את התשובה המלאה, ומסביר מה בדיוק היה חסר.

וזה בדיוק החלק שלא הצלחתי לעשות לבד – לדרג את עצמי בכנות.

להצביע למטה ותראו – הוא כבר קבע מתי נראה את ה-card הזה שוב: התשובה הייתה חלקית, אז

מחר.

מעבר card הבא.

עד 0:25 • 4:35

DEMO

11 DEMO 6 • card שני – הניגוד

11

להדביק את התשובה הטובה המוכנה, לשלוח, ולהצביע על התאריך – ולהשוות בקול לתאריך הקודם.

ועכשיו אחד נוסף – הפעם תשובה מלאה.

להצביע score גבוה, ותראו את התאריך: הוא לא מחר. הוא נדחף הרבה קדימה.

אותה מערכת, שתי תשובות – שני תאריכים שונים לגמרי.

מעבר ללחץ Progress.

להצביע על ה-accuracy ועל הנושאים החלשים.

ובמסך ה-Progress אני מקבלת את התשובה לשאלה שהתחלתי איתה – מה אני צריכה ללמוד היום.

להצביע ה-accuracy שלי, והנושאים שאני הכי נכשלת בהם.

כי לכל card יש state משלו ב-DynamoDB: כל תשובה, כל score, כל תאריך.

מעבר Alt+Tab לטאב 2.

להצביע על traffic, ועל health / errors.

ורציתי מערכת אמיתית, לא רק demo. זה Grafana – להצביע כל ה-requests שעשינו עכשיו נמצאים כאן, live. וכאן error rate וה-health של ה-services.

מעבר Alt+Tab למצגת + להגיד: "ראינו את המערכת מבחוץ. עכשיו מה שקורה מאחורי הקלעים." ואז ↓ פעמיים → שקף 8.

שקף 8.

וכל יום אני מקבלת email שמזכיר לי כמה כרטיסים יש לי היום – כדי שלא אשכח לחזור.

מעבר ↓

להצביע על הרצף העולה.

והאלגוריתם שקובע את התאריכים הוא SM-2: טעיתי – מחר. צדקתי – 1, 6, 15, 39, 97 ימים.

זה Spaced Repetition ו-Active Recall.

מעבר ↓

לא לקרוא את הדיאגרמה. לעקוב עם היד: דפדפן → Sub-Agent → Tutor Agent → S3 → חזרה. לעצור על החץ בין ה-Agents.

אתם רואים את השכבות, אז במקום לעבור תיבה-תיבה – אקח סטודנט אחד ואראה מה קורה לו. הוא נכנס לדפדפן, בוחר נושא, ומעלה PDF.

ה-PDF מגיע ל-Frontend, והמסמך הגולמי נשמר ב-S3 – שם יושבים כל החומרים המקוריים. משם התוכן נשלח ל-Tutor Agent: הוא ממיר אותו לטקסט, ומעבד עם NLP כדי להבין מה כתוב – ומתוך זה מייצר חומר לימוד ושאלות.

וכאן הנקודה המעניינת: ה-Tutor Agent לא עושה הכל בעצמו. כשמשימה דורשת התמחות הוא שולח אותה ל-Sub-Agent המתאים, וה-Sub-Agent מחזיר לו את התוצאה.

כלומר יש כאן communication between agents – כל Agent עושה את מה שהוא מתמחה בו. וכשצריך לגעת במידע, זה עובר דרך MCP: הסוכן מחליט, ה-tool מבצע.

ובסוף ה-Tutor Agent מחזיר לסטודנט את החומר ואת ה-cards.

וזה כל המסלול: מהדפדפן, דרך S3, ל-Tutor Agent, ל-Sub-Agents – וחזרה לסטודנט.

מעבר ↓ פעמיים (מדלגת על שקף 11) → שקף 12.

לעקוב על המסלול Normal → Pending → Firing → Email.

Observability זה לדעת מה קורה למערכת בלי לבהות במסך.

Prometheus אוסף מדדים מה-agent, Alertmanager מחליט מתי להתריע, ו-SNS שולח מייל.

לעקוב ה-agent מפסיק לענות → Pending. וכאן הלב: for: 2m – מתריעים רק אם הבעיה נמשכת שתי דקות, כדי לא לקבל מייל על כל blip.

נמשך → Firing → מייל [FIRING]. חזר → [RESOLVED], וה-alert נסגר לבד.

המערכת מנטרת את עצמה – ורק כשמשוה באמת נשבר, מייל מחכה לי.

מעבר ↓

מהר.

והכל רץ כ-Docker containers על Kubernetes על EC2. את ה-cluster הקמתי בעצמי, את התשתית ב-Terraform, וה-deployment אוטומטי – כל push בונה image ו-Argo CD מסנכרן.

מעבר ↓

19 • 14 • Testing – שקף 14 עד 8:30 • 0:40

להצביע על שתי הקבוצות – Unit ואז Integration – ובסוף על ה-green check של ה-PR.

הכלל בבדיקות: מה שלא בשליטתי – mock. מה שכן שלי – בודקים באמת.

את AWS אני עושה mock – אין טעם שאני אבדוק את אמזון. אבל את SM-2 – הלוגיקה שקובעת מתי card חוזר – אני בודקת על כל מסלול ציון, כי אם היא טועה, כל הלמידה טועה.

בערך 235 unit tests, ומעליהם integration אמיתי: MCP server אמיתי מול DynamoDB Local, מסלול שלם – card נוצר, נשלף בזמן, ומתעדכן אחרי ציון.

וכל PR חייב להיות ירוק.

↓ מעבר

20 • 15 • Working with AI – שקף 15 עד 8:45 • 0:15

שקף 15. משפט אחרון – להאט.

ה-AI כתב הרבה מהקוד – אבל לא קיבל אף החלטה.

עבדתי spec לפני קוד, ואז test first: הבדיקה מגדירה מה נכון, לא המודל.

ה-intervals של SM-2, וה-fail-safe כשהציון לא תקין – אלה היו שלי.

↓ מעבר

21 • 16 • Close – שקף 16 עד 9:00 • 0:15

להאט.

אז זה Recall. התחלתי מבעיה אישית – לומדת ושוכחת – ובניתי מערכת שעושה את זה בשבילי.

ובניתי אותה כמערכת אמיתית, עם deployment ו-monitoring – שגם אחרים יוכלו להשתמש בה.

תודה, אשמח לשאלות.

מעבר אין. להישאר על שקף 16.